

øving_{4e}lk330

Aksel Fosså

September 2026

1 Oppgave 1

$$24 \text{ timer} = 24 \cdot 60 = 1440 \text{ min} \quad (1)$$

$$\frac{1440 \text{ min}}{360^\circ} = 4 \text{ min}/^\circ \quad (2)$$

2 Oppgave 2

$$\Delta t = 4\Delta\lambda \quad (3)$$

$$Cardiff = -3.2 * 4 = -12.8min \quad (4)$$

$$Bristol = -2.6 * 4 = -10.4min \quad (5)$$

$$Greenwich = 0 * 4 = 0 \quad (6)$$

$$Bergen = 5.3 * 4 = 21.2min \quad (7)$$

$$Oslo = 10.8 * 4 = 43.2min \quad (8)$$

$$(9)$$

By	Lengdegrad	Lokal tid i forhold til GMT
Cardiff	-3.2°	GMT -12.8min
Bristol	-2.6°	GMT - 10.4min
Greenwich	0.0°	GMT
Bergen	5.3°	GMT 21.2min
Oslo	10.8°	GMT 43.2min

Table 1:

3 Oppgve 3

Problemet med lokale tider var at alle byene hadde forskjellig klokkeslett. Når tog reiste mellom byene kunne dette føre til forvirring fordi avgang og ankomsttider var oppgit i ulike lokale tider. Personer kunne også bli forvirret hvis de ikke stilte klokken sin når de kom til en ny plass. Dette gjorde også planlegging av togrutter vanskelig siden de hadde mange forskjellige tidsoner og forholde seg til.

4 Oppgave 4

Jeg møtte endre litt på filen siden python liker ikke komma(,) jeg møtte sette inn punktum(.) for alle , for produksjon og forbruk. slik at de ble float.

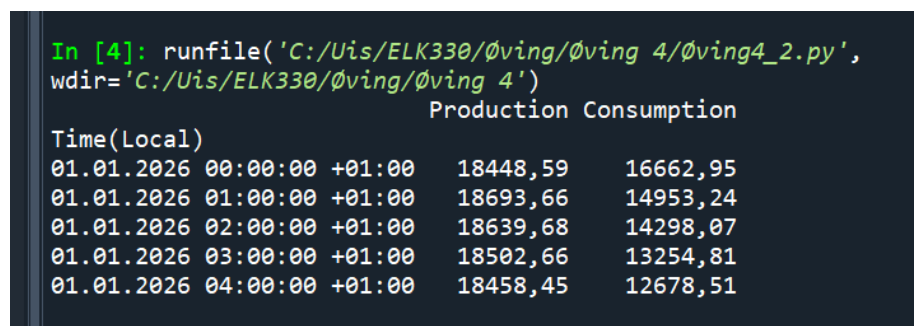


Figure 1: Oppgave 4

hva en DatetimeIndex er?

DatetimeIndex er en indeks som består av dato, klokkeslett og av og til tidzone.

hvorfor tidsstempelet inneholder +01:00?

Tiden inneholder +01:00 siden vi har bestemt at vi bruker tidsonen UTC som er 1 time forand GMT. Norge er med i tidzone UTC.

5 Oppgave 5

Når jeg kjører koden så printer den ut "01.01.2026 00:00:00 +01:00". det er dag, måned, år, timer, min, sek og tidzone som blir vist. Så det er 1.januar 2026 kl.00:00:00 og tidzone +1(UTC)

6 Oppgave 6

Jeg får denne infoen ut

Production 18458,45

Consumption 12678,51

Name: 2026-01-01 03:00:00+00:00, dtype: object

da får jeg akkurat ut all dataen for akkurat det tidspunkte. og dataen er Production og Consumption.

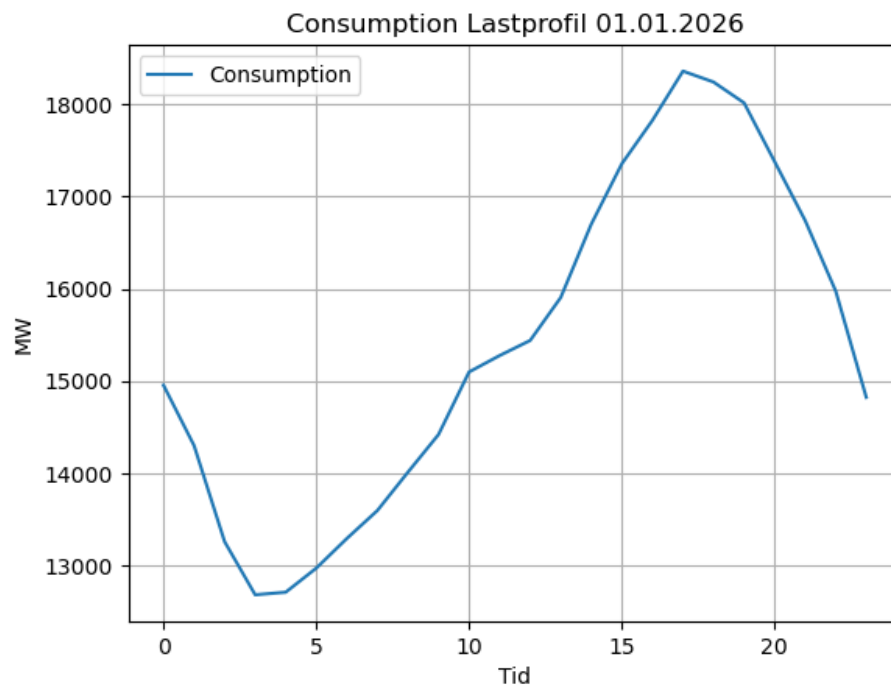


Figure 2: Forbruk 24timer

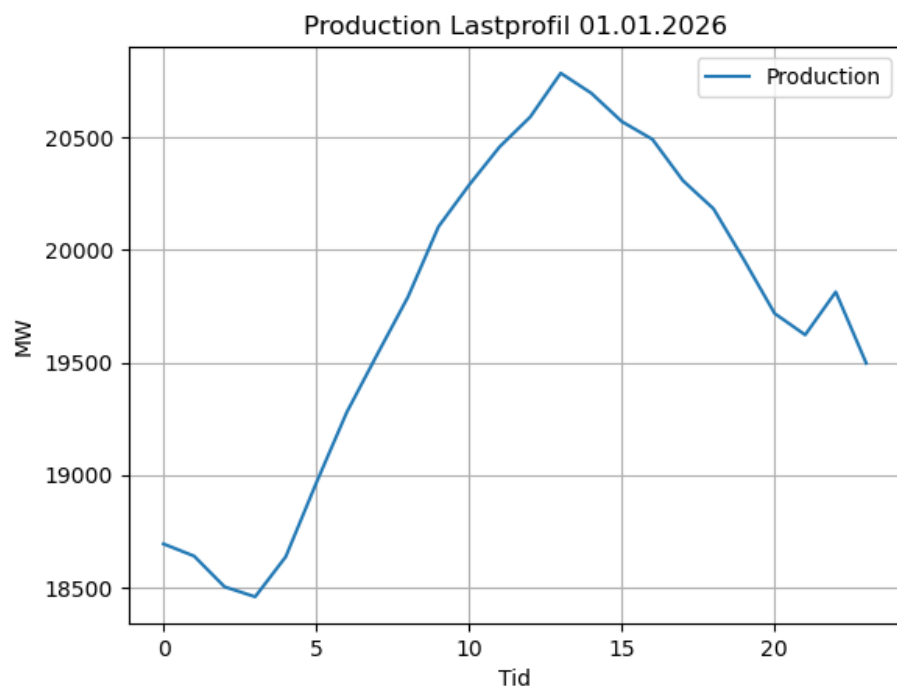


Figure 3: Produksjon 24timer

7 Oppgave 7

Dette er resultatet for den nye Kolonnen.

```
Time(Local)
2025-12-31 23:00:00+00:00    1785.64
2026-01-01 00:00:00+00:00    3740.42
2026-01-01 01:00:00+00:00    4341.61
2026-01-01 02:00:00+00:00    5247.85
2026-01-01 03:00:00+00:00    5779.94
...
2026-05-20 17:00:00+00:00   -5532.16
2026-05-20 18:00:00+00:00   -6238.25
2026-05-20 19:00:00+00:00   -6175.86
2026-05-20 20:00:00+00:00   -5721.79
2026-05-20 21:00:00+00:00   -5400.60
Name: Netto, Length: 3359, dtype: float64
```

Figure 4: Oppgave 7 resultat

8 Oppgave 8

Max=25320.87

Min= 4078.1

Mean=18218.50811253349

```
Pro_max=df["Production"].max()
Pro_min=df["Production"].min()
Pro_mean=df["Production"].mean()

print(Pro_max)
print(Pro_mean)
print(Pro_min)
```

Figure 5: Oppgave 8

9 Oppgave 9

Max= 8068.219999999999. Det skjer 2026-02-22 05:00:00+00:00
Min= -14059.22. Det skjer 2026-04-13 15:00:00+00:00

```
66 #Oppgave 9
67 Nett_max=df["Netto"].max()
68 Nett_min=df["Netto"].min()
69 Nett_max_index = df["Netto"].idxmax()
70 Nett_min_index = df["Netto"].idxmin()
71 print(Nett_max)
72 print(Nett_max_index)
73 print(Nett_min)
74 print(Nett_min_index)
75
```

Figure 6: Oppgave 9

10 Oppgave 10

```
70 #Oppgave 10
71 Sum = df["Production"].sum()
72 print(Sum)
73
74
```

Figure 7: Oppgave 10

Hva representerer resultatet?

Resultater viser den totale produksjone fra 01.01.2026-20.05.2026.

11 Oppgave 11

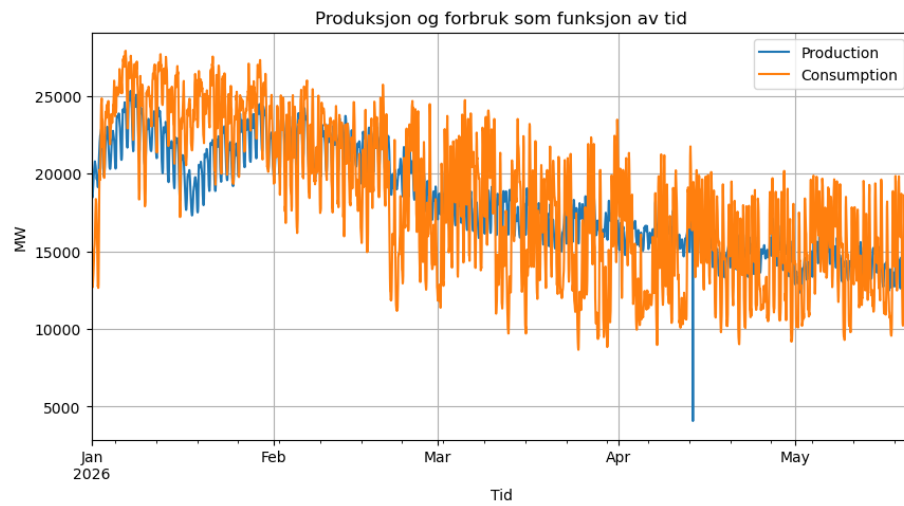


Figure 8: Oppgave 11

12 Oppgave 12

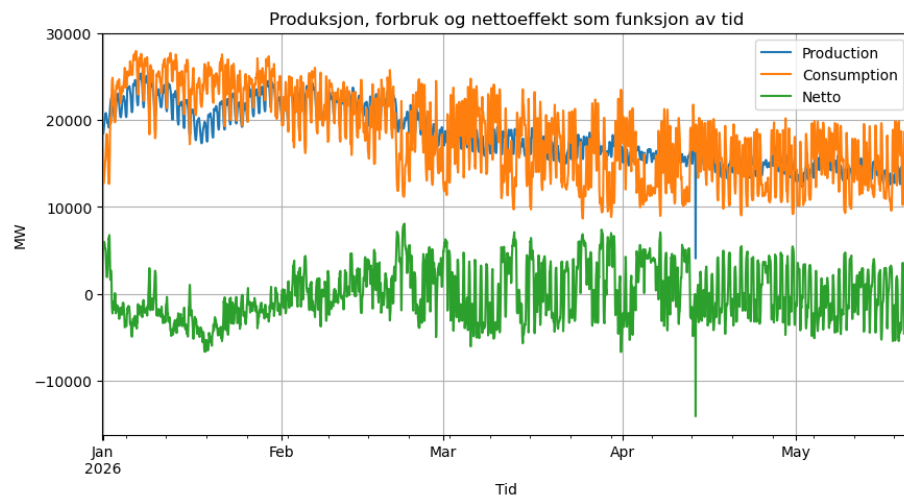


Figure 9: Oppgave 12

13 Oppgave 14

Jeg ser at det er oppgave 14 og ikke 13 men skriver 14 siden det er oppgave nr i øvingen.

Beskriv grafene.

Grafen viser hva produksjon, forbruk og nettoeffekt er for vært tidspunkt fra 1.jan til 20.mai. Den grønne hviser forskjellen på den blå og oransje.

Når er produksjonen høyest?

Den er høyest 2026-01-07 15:00:00

Jeg har funnet det nøyaktig med koden i fig10

Når er forbruket høyest?

Forbruke er høyest 2026-01-06 19:00:00

Har produksjon og forbruk samme mønster?

De har et mønster som er likt til en grad de går opp og ned på veldig like tidspunkter. De ser ut som at det er en sakte nedgang fra januar til mai. Det som er den største forskjellen er at forbruke variere mye mer en produksjonen fra time til time. Vi kan se det med nettoeffekt hvis den er konstant verdi så er mønster likt.

```
97
98 #Oppgave 14
99 Max_Pro_index= df["Production"].idxmax()
100 Max_Con_index= df["Consumption"].idxmax()
101 print(Max_Pro_index)
102 print(Max_Con_index)
103
```

Figure 10: Oppgave 14 kode

14 Oppgave 16

```
Akse1@LAPTOP-VM392920 MINGW64 /c/Uis/ELK330/Øving (master)
$ git log --oneline
7189a02 (HEAD -> master, origin/master) 5
1e65e11 4
e45a362 3
8998d39 2
293027c 1.
074b246 a
bd86e6d a
```

Figure 11: Git bach log

Når jeg så at oppgave 16 var at jeg skulle gjør minst 5 commits så hadde jeg allerede gjort 1 stor for alle bildene og koden. Så det gjorde at commit 2-5 blir bare at jeg committer bilder av at jeg har committet. commit "a" og den andre "a" var når jeg skulle først push det til git, men jeg fikk noe feil med den ene mappen.

```
Akse1@LAPTOP-VM392920 MINGW64 /c/Uis/ELK330/Øving (master)
$ git commit -m "1."
[master 293027c] 1.
15 files changed, 3472 insertions(+), 1 deletion(-)
delete mode 160000 "\303\230ving 4"
create mode 100644 "\303\230ving 4/Oppgave10.png"
create mode 100644 "\303\230ving 4/Oppgave11.png"
create mode 100644 "\303\230ving 4/Oppgave12.png"
create mode 100644 "\303\230ving 4/Oppgave14.png"
create mode 100644 "\303\230ving 4/Oppgave4.png"
create mode 100644 "\303\230ving 4/Oppgave4_kode.png"
create mode 100644 "\303\230ving 4/Oppgave6_Con.png"
create mode 100644 "\303\230ving 4/Oppgave6_pro.png"
create mode 100644 "\303\230ving 4/Oppgave7.png"
create mode 100644 "\303\230ving 4/Oppgave7_kode.png"
create mode 100644 "\303\230ving 4/Oppgave8.png"
create mode 100644 "\303\230ving 4/Oppgave9.png"
create mode 100644 "\303\230ving 4/load_data.csv"
create mode 100644 "\303\230ving 4/\303\230ving4_python.py"
```

Figure 12: Big commit 1

```

Aksel@LAPTOP-VM392920 MINGW64 /c/Uis/ELK330/Øving (master)
$ git commit -m "2"
On branch master
nothing to commit, working tree clean

Aksel@LAPTOP-VM392920 MINGW64 /c/Uis/ELK330/Øving (master)
$ git branch
* master

Aksel@LAPTOP-VM392920 MINGW64 /c/Uis/ELK330/Øving (master)
$ git pull origin master --allow-unrelated-histories
From https://github.com/Akselaksel121/ELK330
 * branch            master       -> FETCH_HEAD
 * [new branch]      master       -> origin/master
Already up to date.

Aksel@LAPTOP-VM392920 MINGW64 /c/Uis/ELK330/Øving (master)
$ git push -u origin master
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 16 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 46.21 KiB | 46.21 MiB/s, done.
Total 4 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/Akselaksel121/ELK330

```

Figure 13: Commit 2

```

Aksel@LAPTOP-VM392920 MINGW64 /c/Uis/ELK330/Øving (master)
$ git add '/c/Uis/ELK330/Øving/Øving 4/Commit2.png'

Aksel@LAPTOP-VM392920 MINGW64 /c/Uis/ELK330/Øving (master)
$ git commit -m "3"
[master e45a362] 3
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 "\303\230ving 4/Commit2.png"

Aksel@LAPTOP-VM392920 MINGW64 /c/Uis/ELK330/Øving (master)
$ git push
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 16 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 63.16 KiB | 31.58 MiB/s, done.
Total 4 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/Akselaksel121/ELK330
8998d39..e45a362 master -> master

```

Figure 14: Commit 3

```

Akse1@LAPTOP-VM392920 MINGW64 /c/Uis/ELK330/Øving (master)
$ git add '/c/Uis/ELK330/Øving/Øving 4/Commit3.png'

Akse1@LAPTOP-VM392920 MINGW64 /c/Uis/ELK330/Øving (master)
$ git commit -m "4"
[master 1e65e11] 4
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 "\303\230ving 4/Commit3.png"

Akse1@LAPTOP-VM392920 MINGW64 /c/Uis/ELK330/Øving (master)
$ git push
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 16 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 52.81 KiB | 52.81 MiB/s, done.
Total 4 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/Akse1aksel21/ELK330
e45a362..1e65e11 master -> master

```

Figure 15: Commit 4

```

Akse1@LAPTOP-VM392920 MINGW64 /c/Uis/ELK330/Øving (master)
$ git add '/c/Uis/ELK330/Øving/Øving 4/Commit4.png'

Akse1@LAPTOP-VM392920 MINGW64 /c/Uis/ELK330/Øving (master)
$ git commit -m "5"
[master 7189a02] 5
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 "\303\230ving 4/Commit4.png"

Akse1@LAPTOP-VM392920 MINGW64 /c/Uis/ELK330/Øving (master)
$ git push
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 16 threads
Compressing objects: 100% (4/4), done.
Writing objects: 100% (4/4), 52.64 KiB | 52.64 MiB/s, done.
Total 4 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/Akse1aksel21/ELK330
1e65e11..7189a02 master -> master

```

Figure 16: Commit 5

15 Refleksjon

En refleksjon på omtrent en halv side:

Hva lærte du om tidsserier?

Jeg har lært i øvingen hvordan jeg skal bruke python/pandas til å lese tid. Jeg har lært hvordan jeg kan bruke tiden til å hente data fra en csv fill for et bestemt tidspunkt. Jeg har lært hvordan jeg kan finne max, min og gjennomsnitt av data som har en tidsserie. Jeg har lært hvordan jeg kan plotte dataen med tanke på tiden.

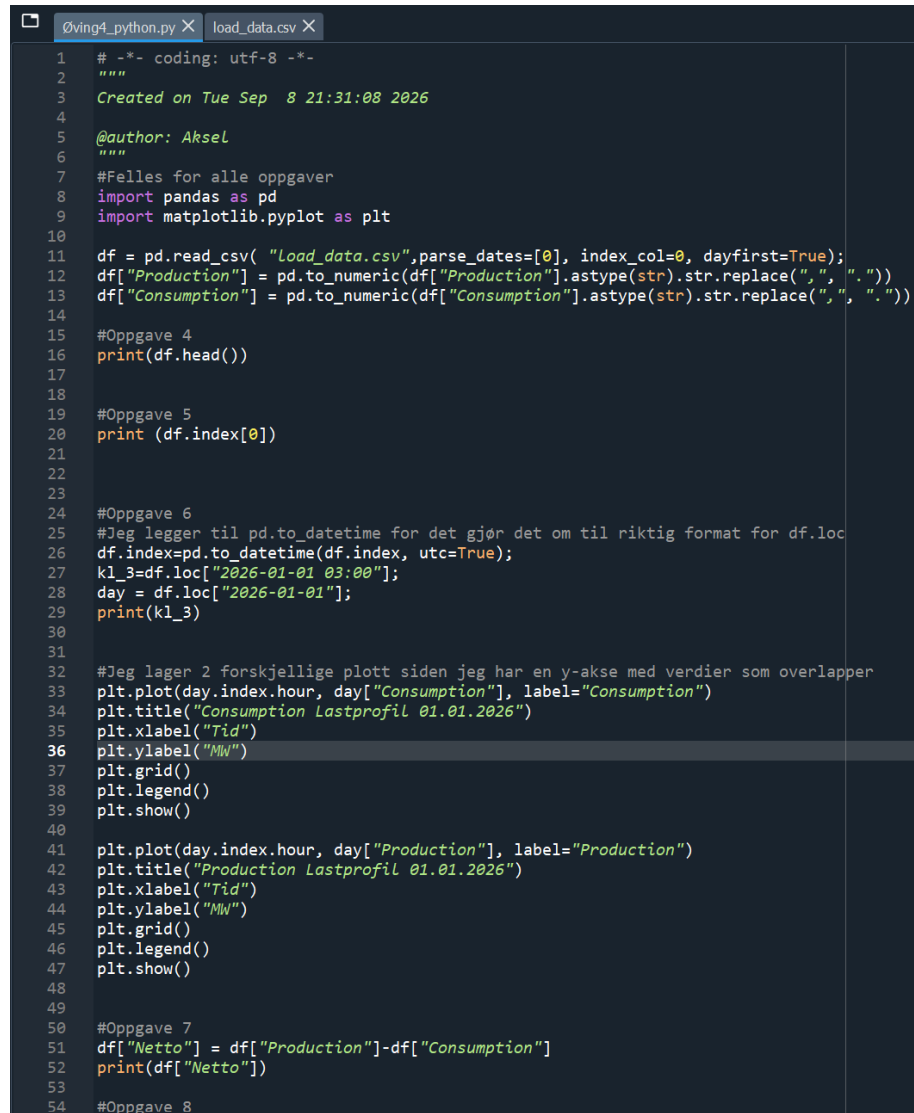
Hva lærte du om Git?

Jeg har lært at git kan være tullete når jeg skal sende ting fra filler til git ved bruk av git bach. Jeg prøvde først å hente git, men det var noe tull så jeg prøvde igjen og så funket det. Når jeg skulle sende mapen til git så var det noe tull der jeg måtte skrive git init, men jeg hadde gjort det i en mape inne i mapen jeg skulle sende så jeg måtte fjerne .git fra den mapen å sende på nytt, men til slutt så fikk jeg alt inn slik jeg ønsket. Hvis jeg får noe mer trening med det så tror jeg det vil funke riktig og at det vil være et bra verktøy.

Hvorfor ble Greenwich Mean Time nødvendig?

Greenwich mean time ble nødvendig fordi ulike steder hadde forskjellige lokale tider. Den ble også nødvedig når jernbanen kom på plass siden det var vanskelig å lage ruter med mange forskjellig tidssoner. Når GMT kom så ble det enklere å kommuniser og plannlegge.

16 Python



```
1 # -*- coding: utf-8 -*-
2 """
3 Created on Tue Sep  8 21:31:08 2026
4
5 @author: Aksel
6 """
7 #Felles for alle oppgaver
8 import pandas as pd
9 import matplotlib.pyplot as plt
10
11 df = pd.read_csv("load_data.csv", parse_dates=[0], index_col=0, dayfirst=True);
12 df["Production"] = pd.to_numeric(df["Production"].astype(str).str.replace(",", "."))
13 df["Consumption"] = pd.to_numeric(df["Consumption"].astype(str).str.replace(",", "."))
14
15 #Oppgave 4
16 print(df.head())
17
18
19 #Oppgave 5
20 print(df.index[0])
21
22
23
24 #Oppgave 6
25 #Jeg legger til pd.to_datetime for det gjør det om til riktig format for df.loc
26 df.index = pd.to_datetime(df.index, utc=True);
27 kl_3 = df.loc["2026-01-01 03:00"];
28 day = df.loc["2026-01-01"];
29 print(kl_3)
30
31
32 #Jeg lager 2 forskjellige plott siden jeg har en y-akse med verdier som overlapper
33 plt.plot(day.index.hour, day["Consumption"], label="Consumption")
34 plt.title("Consumption Lastprofil 01.01.2026")
35 plt.xlabel("Tid")
36 plt.ylabel("MW")
37 plt.grid()
38 plt.legend()
39 plt.show()
40
41 plt.plot(day.index.hour, day["Production"], label="Production")
42 plt.title("Production Lastprofil 01.01.2026")
43 plt.xlabel("Tid")
44 plt.ylabel("MW")
45 plt.grid()
46 plt.legend()
47 plt.show()
48
49
50 #Oppgave 7
51 df["Netto"] = df["Production"] - df["Consumption"]
52 print(df["Netto"])
53
54 #Oppgave 8
```

Figure 17: Python

```

54
55 #Oppgave 8
56
57 Pro_max=df["Production"].max()
58 Pro_min=df["Production"].min()
59 Pro_mean=df["Production"].mean()
60
61 print(Pro_max)
62 print(Pro_mean)
63 print(Pro_min)
64
65
66 #Oppgave 9
67 Nett_max=df["Netto"].max()
68 Nett_min=df["Netto"].min()
69 Nett_max_index = df["Netto"].idxmax()
70 Nett_min_index = df["Netto"].idxmin()
71 print(Nett_max)
72 print(Nett_max_index)
73 print(Nett_min)
74 print(Nett_min_index)
75
76 #Oppgave 10
77 Sum = df["Production"].sum()
78 print(Sum)
79
80 #Oppgave 11
81 df.plot(y=["Production", "Consumption"],figsize=(10,5))
82 plt.title("Produksjon og forbruk som funksjon av tid")
83 plt.xlabel("Tid")
84 plt.ylabel("MW")
85 plt.grid()
86 plt.legend()
87 plt.show()
88
89 #Oppgave 12
90 df.plot(y=["Production", "Consumption", "Netto"],figsize=(10,5))
91 plt.title("Produksjon, forbruk og nettoeffekt som funksjon av tid")
92 plt.xlabel("Tid")
93 plt.ylabel("MW")
94 plt.grid()
95 plt.legend()
96 plt.show()
97
98 #Oppgave 13
99 #Det er ikke en oppgave 13
100
101 #Oppgave 14
102 Max_Pro_index= df["Production"].idxmax()
103 Max_Con_index= df["Consumption"].idxmax()
104 print(Max_Pro_index)
105 print(Max_Con_index)
106

```

Figure 18: Python